

INFO145 - Diseño y Análisis de Algoritmos

9 - Programación Dinámica

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Programación Dinámica (PD)

Es una técnica matemática de diseño para establecer una secuencia de decisiones interrelacionadas a fin de encontrar la combinación óptima de decisiones que resuelven el problema. Desarrollada por Richard Bellman en la década de 1950, se aplica principalmente en problemas de optimización con estructura secuencial.

- Para cada problema específico se debe encontrar la ecuación dinámica que lo resuelve.
- Se comienza resolviendo una pequeña porción del problema, encontrando el óptimo para esta porción.
- Gradualmente expande el tamaño del problema, encontrando el óptimo a partir de porciones menores ya solucionadas.

... Programación Dinámica

- “Programación” se refiera a construir una Tabla que programe lo que se debe hacer (un *scheduler*) y no a construir un programa computacional.
- Los algoritmos para PD resuelven cada subproblema solo una vez, **guardando las respuestas** en una tabla que se reutilizará cada vez que el mismo subproblema sea requerido por etapas posteriores.

En general, la solución óptima del problema global se construye combinando soluciones óptimas de sus subproblemas. Esto se rige por el principio de optimalidad de Bellman, que establece que las decisiones futuras deben ser independientes de las anteriores

... Programación Dinámica

Los problemas que se pueden resolver con PD son aquellos que:

- Generalmente son problemas de optimización con muchas soluciones posibles.
- El problema puede ser redefinido en etapas secuenciales.
- Encontrar el óptimo de cada etapa depende exclusivamente de los datos actuales y locales a la etapa, y de los óptimos ya encontrados en etapas preliminares pero nunca de cálculos de etapas futuras.
- Son similares a los solubles con Divide y Vencerás (DV), ya que primero se resuelven problemas pequeños y sus soluciones se combinan para obtener la solución final al problema.
- Una diferencia con DV es que en PD los subproblemas si pueden sobrelaparse en cambio en DV se hace una partición de la entrada o problema.
- También se usa para acelerar un algoritmo a costo de espacio extra.

... Programación Dinámica

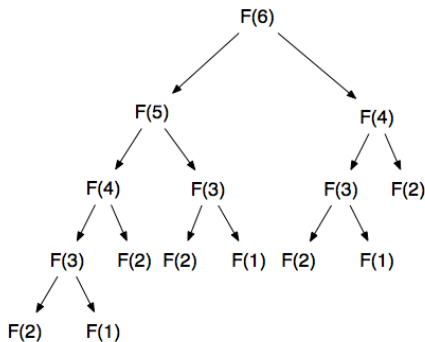
Los pasos a seguir en PD, se pueden resumir en:

1. Definir la estructura de una solución óptima
2. Definir recursivamente el valor de una solución óptima mediante una ecuación matemática.
3. Calcular el valor de las soluciones óptimas locales.
4. Construir el óptimo global:
El enfoque Bottom-Up lo construye a partir de las soluciones calculadas.
El enfoque Top-Down llena iterativamente una tabla con soluciones desde los subproblemas más pequeños.

Ejemplo - Fibonacci

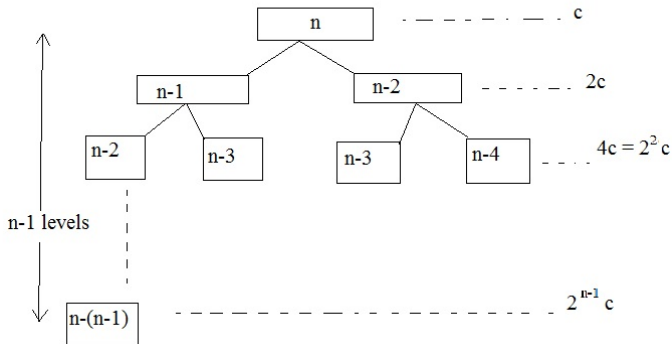
```
fibonacci( $n$ ){  
  if ( $n \leq 2$ ) then  
    return 1  
  return fibonacci( $n - 2$ ) + fibonacci( $n - 1$ )  
}
```

Tiempo: $T(n) = T(n - 1) + T(n - 2) + c$



Fibonacci - Recurrence Tree

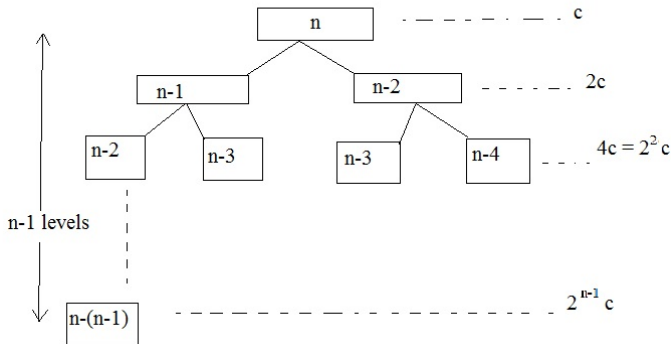
$$T(n) = T(n-1) + T(n-2) + c$$



$$T(n) = c + 2c + 2^2c + 2^3c + \dots + 2^{n-1}c$$

Fibonacci - Recurrence Tree

$$T(n) = T(n-1) + T(n-2) + c$$



$$T(n) = c + 2c + 2^2c + 2^3c + \dots + 2^{n-1}c = \sum_{k=0}^{n-1} c2^k = c(2^n - 1) = O(2^n)$$

Fibonacci con PD

Vamos a usar un arreglo $F[1..N]$ para guardar los cálculos ya realizados y no tener que recalcular invocando nuevamente a la recursividad.

Fibonacci con PD

Vamos a usar un arreglo $F[1..N]$ para guardar los cálculos ya realizados y no tener que recalcular invocando nuevamente a la recursividad.

Input: Arreglo $F[1..N]$, con $N \geq 2$, $F[1] = F[2] = 1$, y $F[i] = 0 \forall 2 < i \leq N$, y el n de la serie

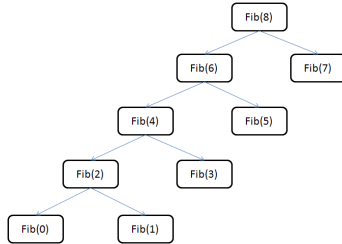
Output: n -ésimo término de la serie

```
fibonacciPD( $F[1..N]$ ,  $n$ ) {  
    if ( $n \leq N$ ) then {  
        if ( $F[n] > 0$ ) then  
            return  $F[n]$   
        if ( $F[n-2] = 0$ ) then  
             $F[n-2] = \text{fibonacciPD}(F, n-2)$   
        if ( $F[n-1] = 0$ ) then  
             $F[n-1] = \text{fibonacciPD}(F, n-1)$   
         $F[n] = F[n-1] + F[n-2]$   
        return  $F[n]$   
    }  
    return fibonacciPD( $F, n-2$ ) + fibonacciPD( $F, n-1$ )  
}
```

¿Cómo queda el árbol de recurrencia ahora?

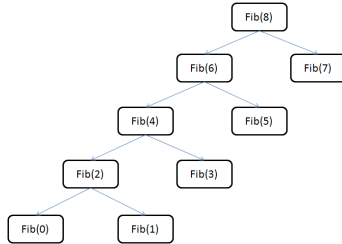
FibonacciPD - Recurrence Tree

Si hay espacio suficiente, $n \leq N$, entonces el árbol queda:



FibonacciPD - Recurrence Tree

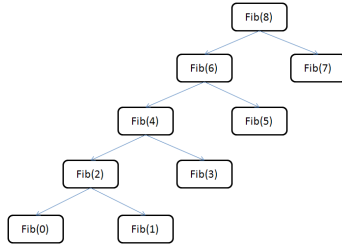
Si hay espacio suficiente, $n \leq N$, entonces el árbol queda:



$$T(n) = c + c + c + \dots + c = c(n + 1) = O(n)$$

FibonacciPD - Recurrence Tree

Si hay espacio suficiente, $n \leq N$, entonces el árbol queda:



$$T(n) = c + c + c + \dots + c = c(n + 1) = O(n)$$

Ejercicios:

- 1- Elimine la recursividad del código anterior, suponga que $N = n$.
- 2- En su nuevo código, ¿Puede prescindir del arreglo auxiliar, obviamente sin tener que recalcular los términos de la serie (aunque esto ya no sería PD)?

Rod Cutting Problem

Definición el Problema:

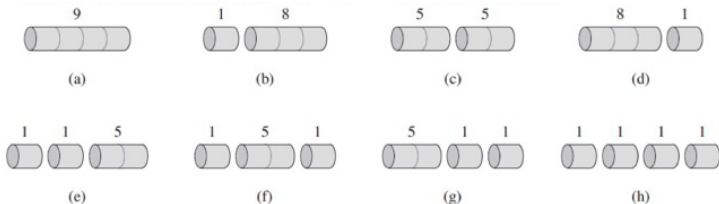
- Se tiene una barra o cilindro de acero de largo n , la cual se desea vender, ya sea entera o en varias partes a fin de maximizar los ingresos de la venta.
- Existe una tabla de precios $P[1 \dots n]$; tal que $P[k]$ es el precio en dolares de vender un segmento de barra de largo k (suponga pulgadas).
- Entonces el problema consiste en determinar como cortar (o no cortar) la barra para generar la mayor utilidad en su venta.

Rod Cutting Problem - Ejemplo

Suponga que la barra mide 4 pulgadas ($n = 4$) y que los cortes se restringen a pulgadas enteras, con la siguiente tabla de precios:

$$P[1 \dots 4] = \{1, 5, 8, 9\}$$

Entonces podríamos tener las siguientes configuraciones para sus cortes:



Así, el óptimo es vender la barra en dos trozos de 5 pulgadas cada uno. Note además que existen 2^{n-1} formas posibles de cortar la barra.

Rod Cutting - Estructura

- Si la decisión óptima es cortar la barra de largo n en k trozos, el ingreso óptimo viene dado por:

$$r(n) = \max\{P[n], r(1) + r(n-1), r(2) + r(n-2), \dots, r(n-1) + r(1)\}$$

- Esto es, probar todas las formas posibles y seleccionar la de mayor beneficio; donde $P[n]$ es el precio de un segmento de largo n , $r(i) + r(n-i)$ es el óptimo para un corte de largo i + el óptimo para una barra de largo $n-i$, segmentos que a su vez se pueden volver a segmentar.
- Así podemos definir nuestra **Ecuación de Recurrencia Dinámica (ERD)** como:

$$r(n) = \max\{P[i] + r(n-i)\}, \forall i = 1, 2, \dots, n$$

Algoritmo Exhaustivo

El siguiente pseudocódigo resuelve el problema mediante **Búsqueda Exhaustiva**
o **Fuerza Bruta**:

```
CUT-ROD( $P, n$ ){  
    if( $n == 0$ )  
        return 0  
     $q = -\infty$   
    for( $i = 1$  to  $n$ )  
         $q = \max(q, P[i] + \text{CUT-ROD}(P, n - i))$   
    return  $q$   
}
```

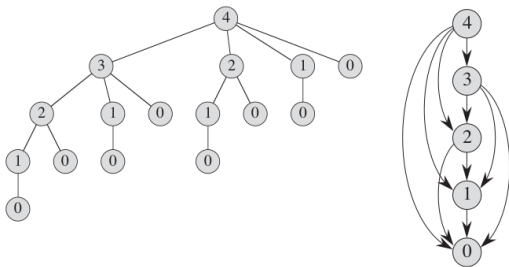
Algoritmo Exhaustivo

El siguiente pseudocódigo resuelve el problema mediante **Búsqueda Exhaustiva**
o **Fuerza Bruta**:

```
CUT-ROD( $P, n$ ){  
    if( $n == 0$ )  
        return 0  
     $q = -\infty$   
    for( $i = 1$  to  $n$ )  
         $q = \max(q, P[i] + \text{CUT-ROD}(P, n - i))$   
    return  $q$   
}
```

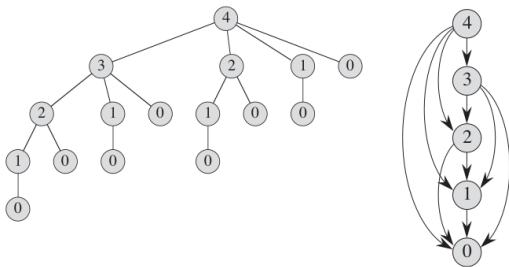
Note que el tiempo de esta solución es de $O(2^n)$, ya que prueba todas las combinaciones posibles para cortar la barra.

Rod Cutting - Algoritmo Exhaustivo - Recurrence Tree



- El grafo de la derecha se llama **Grafo de Subproblemas**, se usa para visualizar las relaciones de gerarquía entre los subproblemas involucrados en la solución.
- En el árbol de recurrencia (izquierda) hay exactamente $2^4 = 16$ nodos. Con este podemos establecer su ecuación de recurrencia, con $T(0) = 1$:
$$T(n) = 1 + T(0) + T(1) + \dots + T(n-1)$$

Rod Cutting - Algoritmo Exhaustivo - Recurrence Tree



- El grafo de la derecha se llama **Grafo de Subproblemas**, se usa para visualizar las relaciones de gerarquía entre los subproblemas involucrados en la solución.
- En el árbol de recurrencia (izquierda) hay exactamente $2^4 = 16$ nodos. Con este podemos establecer su ecuación de recurrencia, con $T(0) = 1$:
$$T(n) = 1 + T(0) + T(1) + \dots + T(n-1) = 1 + 2^0 + 2^1 + \dots + 2^{n-1} = 2^n$$

Top Down Rod Cutting con PD

Apliquemos PD para resolver el problema con la técnica *top-down*, donde usaremos un arreglo $R[1..n]$ para guardar las ganancias óptimas ya calculadas.

TopDown-CUT-ROD(P, n) {

 Sea $R[0 \dots n]$ un arreglo de $n + 1$ enteros

 for($i = 0$ to n)

$R[i] = -\infty$

 return TopDown-CUT-ROD-Rec(P, n, R)

}

TopDown-CUT-ROD-Rec(P, n, R) {

 if($R[n] \geq 0$) return $R[n]$

 if($n == 0$) $q = 0$

 else {

$q = -\infty$

 for($i = 1$ to n)

$q = \max(q, P[i] + \text{TopDown-CUT-ROD-Rec}(P, n - i, R))$

 }

$R[n] = q$

 return q

}

Bottom Up Rod Cutting con PD

En PD resulta, por lo general, ser más fácil utilizar *bottom-up* que *top-down*, ya que es más natural comenzar resolviendo el problema en pequeñas partes y luego ampliar hasta cubrir todo el rango pedido.

```
BottomUp-CUT-ROD( $P, n$ ){  
    Sea  $R[0 \dots n]$  un arreglo de  $n + 1$  enteros  
     $R[0] = 0$   
    for( $j = 1$  to  $n$ ){  
         $q = -\infty$   
        for( $i = 1$  to  $j$ )  
             $q = \max(q, P[i] + R[j - i])$   
         $R[j] = q$   
    }  
    return  $R[n]$   
}
```

Bottom Up Rod Cutting con PD

En PD resulta, por lo general, ser más fácil utilizar *bottom-up* que *top-down*, ya que es más natural comenzar resolviendo el problema en pequeñas partes y luego ampliar hasta cubrir todo el rango pedido.

```
BottomUp-CUT-ROD( $P, n$ ) {  
    Sea  $R[0 \dots n]$  un arreglo de  $n + 1$  enteros  
     $R[0] = 0$   
    for( $j = 1$  to  $n$ ) {  
         $q = -\infty$   
        for( $i = 1$  to  $j$ )  
             $q = \max(q, P[i] + R[j - i])$   
         $R[j] = q$   
    }  
    return  $R[n]$   
}
```

Ejercicios.

Compruebe que el costo de ambos algoritmos es $O(n^2)$. Como tip para la técnica TopDown, puede dibujar el árbol de recurrencia y determinar el costo.

Reconstrucción de la Solución Óptima

- Note que ambos algoritmos entregan el valor de la venta óptima de la barra de largo n , pero no especifican de qué forma hay que cortar la barra.
- Lo común es que no solo obtengamos el monto óptimo, sino que además se nos diga como debemos cortar la barra.
- La solución propuesta es utilizar (nuevamente) memoria adicional para ir “recordando” en qué partes fue cortada la barra al hallar cada óptimo local para los diferentes largos.
- El algoritmo **Extended-BottomUp-CUT-ROD**, crea los arreglos $R[0 \dots n]$ y $S[1 \dots n]$, donde $S[i]$ almacena el largo del corte óptimo para una barra de largo i .
- Luego el algoritmo **Print-CUT-ROD-Solution** se encarga de imprimir los largos de los $k \leq n$ cortes de la barra.

Guardando la Solución Óptima

```
Extended-BottomUp-CUT-ROD( $P, n$ ){  
    Sea  $R[0 \dots n]$  y  $S[1 \dots n]$  arreglos de enteros  
     $R[0] = 0$   
    for( $j = 1$  to  $n$ ){  
         $q = -\infty$   
        for( $i = 1$  to  $j$ ){  
            if( $P[i] + R[j - i] > q$ ){  
                 $q = P[i] + R[j - i]$   
                 $S[j] = i$   
            }  
        }  
         $R[j] = q$   
    }  
}  
return ( $R, S$ )  
}
```

Imprimiendo la Solución Óptima

```
Print-CUT-ROD-Solution( $P, n$ ){  
    ( $R, S$ ) = Extended-BottomUp-CUT-ROD( $P, n$ )  
    while( $n > 0$ ){  
        print( $S[n]$ )  
         $n = n - S[n]$   
    }  
}
```

Ejercicios.

Considere la siguiente tabla de precios:

Largos: 1 2 3 4 5 6 7 8 9 10

Precios: 1 5 8 9 10 17 17 20 24 30

Haga la traza (o tabla de ejecución) y determine el óptimo para una barra de largo $n = 10$ con ambos algoritmos y entregue cuál es la configuración óptima.